DATA 260 — HOMEWORK 8

Kafka Request/Response and Event-Driven Agent Pipelines

Vedant Vartak

This report documents the implemented Kafka RPC service the three-process agent pipeline and four live failure experiments. Every screenshot below was generated from the saved raw run evidence in **reports/hw08/raw**.

Required item	Submitted value
SID4 / PREFIX / PORT_BASE	3539 / s3539 / 8839
SEED / VERIFY_SEED / DOMAIN_ID	3539 / 263539 / 3
Hardware	Apple M4 MacBook Air — 10 cores — 16 GB RAM
Local model	qwen3:8b
Repository	https://github.com/vedantvartakwork/data260-3539
Implementation and evidence commit	66ef001731bf57b75530c0c1da2d55ca8153e0a1
Submission tag	hw8

Verification summary

Automated verification passed 5/5 checks. The suite includes 18 named unit tests plus direct inspection of the live Kafka failure-experiment JSON.

Collaborators: Sbnikitha and supriyaselvanganesan are the course collaborators associated with this repository.

1. Requirement map and architecture

The implementation uses Kafka in KRaft mode through Docker Compose. FastAPI acts as the client-facing process for the entity request/response flow. The recall worker owns an in-memory dictionary. Planner Writer and Reviewer are separate processes with independent consumer groups.

Assignment requirement	Implementation / evidence
Entity request/response	code/hw8/rpc.py service.py worker.py plus live flow
Three-process agent pipeline	agent_worker.py and planner/writer/reviewer.py plus live stage logs
Duplicate delivery	correlation-ID response cache plus live replay
Consumer restart	manual commit after publish plus live stop/start
Poison message	three attempts then s3539_dead_letter plus live continuation
Ordering	20 messages unkeyed vs keyed with two live Writers
Raw and verification artifacts	reports/hw08/raw and verification.json

Kafka topics

Topic	Partitions	Purpose
s3539_entity_requests	1	CREATE_RECALL and GET_RECALL requests
s3539_entity_responses	1	Correlated entity responses
s3539_inbox	1	User questions for Planner
s3539_tasks	2	Planner output and Writer scaling/ordering experiment
s3539_drafts	1	Writer drafts for Reviewer
s3539_final	1	Approved final response
s3539_dead_letter	1	Poison records after bounded retries

2. Reproducible Docker and Kafka setup

The Compose file starts Kafka FastAPI one recall worker Planner two Writer instances and Reviewer. The initialization container creates the seven exact topics and gives s3539_tasks two partitions. The stack is reproducible with make hw8-up and removed with make hw8-down.

```
services:
  kafka:
    image: apache/kafka:4.0.0
    container_name: s3539-kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: INTERNAL://:29092,EXTERNAL://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:29092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:29092 >/dev/null 2
      interval: 5s
      timeout: 5s
      retries: 30

...
      KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    depends_on:
      topic-init:
        condition: service_completed_successfully

  writer-2:
    build:
      context: .
      dockerfile: Dockerfile.hw8
    command: python -m code.hw8.writer
    environment:
      KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    depends_on:
      topic-init:
        condition: service_completed_successfully

  reviewer:
    build:
      context: .
      dockerfile: Dockerfile.hw8
    command: python -m code.hw8.reviewer
    environment:
      KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    depends_on:
      topic-init:
        condition: service_completed_successfully

```

2.1 Live container evidence



Figure 1. Live Docker Compose state. Kafka is healthy all six application processes are running and topic initialization exited successfully.

The raw command output is saved as reports/hw08/raw/docker_ps.txt. Service logs and exact topic creation output are saved in kafka_service_logs.txt.

3. Part 1 — Kafka request/response

The client creates a UUID correlation_id includes reply_to operation and data and publishes JSON to s3539_entity_requests. A background response consumer resolves the matching Future only when the returned correlation_id matches. The worker validates every field performs CREATE_RECALL or GET_RECALL and publishes a clean success or error response.

3.1 Domain validation and idempotent worker

```
31      def validate_create(data: Any) -> str | None:
32          if not isinstance(data, dict):
33              return "data must be an object"
34          required_strings = (
35              "product_name",
36              "recall_code",
37              "brand_name",
38              "submitter_email",
39              "category",
40              "recall_details",
41          )
42          for field in required_strings:
43              if not isinstance(data.get(field), str) or not data[field].strip():
44                  return f"{field} is required"
45          if not EMAIL_RE.fullmatch(data["submitter_email"].strip()):
46              return "submitter_email must be a valid email address"
47          if data["category"] not in VALID_CATEGORIES:
48              return "category is invalid"
49          units = data.get("units_affected")
50          if isinstance(units, bool) or not isinstance(units, int) or units <= 0:
51              return "units_affected must be a positive integer"
52          if data.get("terms_accepted") is not True:
53              return "terms_accepted must be true"
54          return None
...
56      def process(self, message: Any) -> dict[str, Any]:
57          if not isinstance(message, dict):
58              return self._error("", "Request must be a JSON object")
59          correlation_id = message.get("correlation_id")
60          if not isinstance(correlation_id, str) or not correlation_id:
61              return self._error("", "correlation_id is required")
62          if correlation_id in self.responses_by_correlation:
63              replay = deepcopy(self.responses_by_correlation[correlation_id])
64              replay["duplicate"] = True
65              return replay
66          if not isinstance(message.get("reply_to"), str) or not message["reply_to"]:
67              return self._error(correlation_id, "reply_to is required")
68
69          operation = message.get("operation")
70          data = message.get("data")
71          if operation == "CREATE_RECALL":
72              error = self.validate_create(data)
73              if error:
74                  response = self._error(correlation_id, error)
75              else:
76                  entity_id = str(uuid.uuid4())
77                  entity = {"entity_id": entity_id, **deepcopy(data)}
78                  self.entities[entity_id] = entity
79                  response = {
80                      "correlation_id": correlation_id,
81                      "success": True,
82                      "entity_id": entity_id,
83                      "message": "Recall created",
84                  }
85          elif operation == "GET_RECALL":
86              entity_id = data.get("entity_id") if isinstance(data, dict) else None
87              if not entity_id:
88                  response = self._error(correlation_id, "entity_id is required")
89              elif entity_id not in self.entities:
90                  response = self._error(correlation_id, "Entity not found")
91              else:
```

```
                response = {
                    "correlation_id": correlation_id,
                    "success": True,
                    "entity": deepcopy(self.entities[entity_id]),
                }
        else:
            response = self._error(correlation_id, "Unknown operation")

        self.responses_by_correlation[correlation_id] = deepcopy(response)
        return response
```

3.2 Correlation-aware RPC client

```
    async def _consume_responses(self) -> None:
        async for record in self.consumer:
            response = record.value
            correlation_id = response.get("correlation_id")
            print(
                f"[FASTAPI RECEIVE] topic={record.topic} partition={record.partition} "
                f"offset={record.offset} correlation_id={correlation_id} "
                f"message={json.dumps(response, sort_keys=True)}",
                flush=True,
            )
            future = self.pending.pop(correlation_id, None)
            if future and not future.done():
                future.set_result(response)

    async def request(
        self,
        operation: str,
        data: dict[str, Any],
        *,
        correlation_id: str | None = None,
    ) -> dict[str, Any]:
        if not self.producer:
            raise RuntimeError("Kafka RPC is not started")
        correlation_id = correlation_id or str(uuid.uuid4())
        message = {
            "correlation_id": correlation_id,
            "reply_to": settings.entity_responses,
            "operation": operation,
            "data": data,
        }
        loop = asyncio.get_running_loop()
        future: asyncio.Future[dict[str, Any]] = loop.create_future()
        self.pending[correlation_id] = future
        metadata = await self.producer.send_and_wait(settings.entity_requests, message)
        print(
            f"[FASTAPI SEND] topic={metadata.topic} partition={metadata.partition} "
            f"offset={metadata.offset} correlation_id={correlation_id} "
            f"message={json.dumps(message, sort_keys=True)}",
            flush=True,
        )
        try:
            return await asyncio.wait_for(future, timeout=self.timeout_seconds)
        except TimeoutError:
            self.pending.pop(correlation_id, None)
            return {
                "correlation_id": correlation_id,
                "success": False,
                "error": "Kafka response timed out",
            }
```

3.3 Manual worker response and offset commit

```
    producer = AIOKafkaProducer(
        bootstrap_servers=settings.kafka_bootstrap,
        value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    )
    await consumer.start()
    await producer.start()
    print("Vedant Vartak - Consumer is ready and listening...", flush=True)
    print(f"[WORKER] request_topic={settings.entity_requests}", flush=True)
    try:
        async for record in consumer:
            message = record.value
            correlation_id = message.get("correlation_id", "")
            print(
                f"[WORKER RECEIVE] topic={record.topic} partition={record.partition} "
                f"offset={record.offset} correlation_id={correlation_id} "
                f"message={json.dumps(message, sort_keys=True)}",
                flush=True,
            )
            response = service.process(message)
            reply_to = message.get("reply_to", settings.entity_responses)
            metadata = await producer.send_and_wait(reply_to, response)
            print(
                f"[WORKER REPLY] topic={metadata.topic} partition={metadata.partition} "
                f"offset={metadata.offset} correlation_id={correlation_id} "
                f"response={json.dumps(response, sort_keys=True)}",
                flush=True,
            )
            await consumer.commit()
            print(f"[WORKER COMMIT] correlation_id={correlation_id} after_success=true", flush=True)
    finally:
        await consumer.stop()
        await producer.stop()


if __name__ == "__main__":
    asyncio.run(run_worker())
```

The record is committed only after the response is successfully sent. This is the at-least-once boundary used again in the failure experiment.

3.4 Complete live CREATE then GET run

```
CREATE_RECALL THEN GET_RECALL
Vedant Vartak — DATA 260 Homework 8
timestamp_utc=2026-10-27T05:13:28.012490+00:00

Vedant Vartak — DATA 260 Homework 8 complete Kafka flow
CLIENT SEND CREATE_RECALL {"correlation_id": "d4db56c8-70ba-4ca5-8ccc-4d2debee7826", "data": {"brand_name":
  "Campus Pantry Evidence", "category": "Packaged Foods", "product_name": "HW8 Evidence Granola Bars",
  "recall_code": "REC-3539-80001", "recall_details": "Packages may contain an undeclared peanut allergen.",
  "submitter_email": "student@example.edu", "terms_accepted": true, "units_affected": 3539}, "operation":
  "CREATE_RECALL"}
CLIENT RECEIVE CREATE_RECALL {"correlation_id": "d4db56c8-70ba-4ca5-8ccc-4d2debee7826", "entity_id":
  "979999a2-fa30-4adf-bb4b-6c61c07fb9e3", "message": "Recall created", "success": true}
CLIENT SEND GET_RECALL {"correlation_id": "6383431c-186a-45b1-b17c-9279ee426c86", "data": {"entity_id":
  "979999a2-fa30-4adf-bb4b-6c61c07fb9e3"}, "operation": "GET_RECALL"}
CLIENT RECEIVE GET_RECALL {"correlation_id": "6383431c-186a-45b1-b17c-9279ee426c86", "entity": {"brand_name":
  "Campus Pantry Evidence", "category": "Packaged Foods", "entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3",
  "product_name": "HW8 Evidence Granola Bars", "recall_code": "REC-3539-80001", "recall_details": "Packages may
  contain an undeclared peanut allergen.", "submitter_email": "student@example.edu", "terms_accepted": true,
  "units_affected": 3539}, "success": true}
RETRIEVED ENTITY {"brand_name": "Campus Pantry Evidence", "category": "Packaged Foods", "entity_id":
  "979999a2-fa30-4adf-bb4b-6c61c07fb9e3", "product_name": "HW8 Evidence Granola Bars", "recall_code":
  "REC-3539-80001", "recall_details": "Packages may contain an undeclared peanut allergen.", "submitter_email":
  "student@example.edu", "terms_accepted": true, "units_affected": 3539}
PASS CREATE_RECALL -> GET_RECALL and both correlation IDs were preserved
```

Figure 2. One live call creates a UUID recall and a second call retrieves that same UUID. Each response preserves its own correlation ID.

The input includes typed integer units accepted terms a valid email category product details and recall code. Unknown operations missing fields invalid units and invalid email values return explicit errors; the test suite covers each case.

```
KAFKA RPC SEND -> RECEIVE -> REPLY -> CLIENT
Vedant Vartak - DATA 260 Homework 8
recall-worker-1  | [WORKER RECEIVE] topic=s3539_entity_requests partition=0 offset=6
  correlation_id=d4db56c8-70ba-4ca5-8ccc-4d2debee7826 message={"correlation_id":
  "d4db56c8-70ba-4ca5-8ccc-4d2debee7826", "data": {"brand_name": "Campus Pantry Evidence", "category": "Packaged
   Foods", "product_name": "HW8 Evidence Granola Bars", "recall_code": "REC-3539-80001", "recall_details":
  "Packages may contain an undeclared peanut allergen.", "submitter_email": "student@example.edu",
  "terms_accepted": true, "units_affected": 3539}, "operation": "CREATE_RECALL", "reply_to":
  "s3539_entity_responses"}
recall-worker-1  | [WORKER REPLY] topic=s3539_entity_responses partition=0 offset=6
  correlation_id=d4db56c8-70ba-4ca5-8ccc-4d2debee7826 response={"correlation_id":
  "d4db56c8-70ba-4ca5-8ccc-4d2debee7826", "entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3", "message":
  "Recall created", "success": true}
recall-worker-1  | [WORKER COMMIT] correlation_id=d4db56c8-70ba-4ca5-8ccc-4d2debee7826 after_success=true
recall-worker-1  | [WORKER RECEIVE] topic=s3539_entity_requests partition=0 offset=7
  correlation_id=6383431c-186a-45b1-b17c-9279ee426c86 message={"correlation_id":
  "6383431c-186a-45b1-b17c-9279ee426c86", "data": {"entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3"},
  "operation": "GET_RECALL", "reply_to": "s3539_entity_responses"}
recall-worker-1  | [WORKER REPLY] topic=s3539_entity_responses partition=0 offset=7
  correlation_id=6383431c-186a-45b1-b17c-9279ee426c86 response={"correlation_id":
  "6383431c-186a-45b1-b17c-9279ee426c86", "entity": {"brand_name": "Campus Pantry Evidence", "category":
  "Packaged Foods", "entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3", "product_name": "HW8 Evidence Granola
  Bars", "recall_code": "REC-3539-80001", "recall_details": "Packages may contain an undeclared peanut
  allergen.", "submitter_email": "student@example.edu", "terms_accepted": true, "units_affected": 3539},
  "success": true}
recall-worker-1  | [WORKER COMMIT] correlation_id=6383431c-186a-45b1-b17c-9279ee426c86 after_success=true
fastapi-1        | [FASTAPI SEND] topic=s3539_entity_requests partition=0 offset=6
  correlation_id=d4db56c8-70ba-4ca5-8ccc-4d2debee7826 message={"correlation_id":
  "d4db56c8-70ba-4ca5-8ccc-4d2debee7826", "data": {"brand_name": "Campus Pantry Evidence", "category": "Packaged
   Foods", "product_name": "HW8 Evidence Granola Bars", "recall_code": "REC-3539-80001", "recall_details":
  "Packages may contain an undeclared peanut allergen.", "submitter_email": "student@example.edu",
  "terms_accepted": true, "units_affected": 3539}, "operation": "CREATE_RECALL", "reply_to":
  "s3539_entity_responses"}
fastapi-1        | [FASTAPI RECEIVE] topic=s3539_entity_responses partition=0 offset=6
  correlation_id=d4db56c8-70ba-4ca5-8ccc-4d2debee7826 message={"correlation_id":
  "d4db56c8-70ba-4ca5-8ccc-4d2debee7826", "entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3", "message":
  "Recall created", "success": true}
fastapi-1        | [FASTAPI SEND] topic=s3539_entity_requests partition=0 offset=7
  correlation_id=6383431c-186a-45b1-b17c-9279ee426c86 message={"correlation_id":
  "6383431c-186a-45b1-b17c-9279ee426c86", "data": {"entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3"},
  "operation": "GET_RECALL", "reply_to": "s3539_entity_responses"}
fastapi-1        | [FASTAPI RECEIVE] topic=s3539_entity_responses partition=0 offset=7
  correlation_id=6383431c-186a-45b1-b17c-9279ee426c86 message={"correlation_id":
  "6383431c-186a-45b1-b17c-9279ee426c86", "entity": {"brand_name": "Campus Pantry Evidence", "category":
  "Packaged Foods", "entity_id": "979999a2-fa30-4adf-bb4b-6c61c07fb9e3", "product_name": "HW8 Evidence Granola
  Bars", "recall_code": "REC-3539-80001", "recall_details": "Packages may contain an undeclared peanut
  allergen.", "submitter_email": "student@example.edu", "terms_accepted": true, "units_affected": 3539},
  "success": true}
```

Figure 3. The same correlation IDs appear in the FastAPI send the Worker receive and reply and the FastAPI receive for both CREATE_RECALL and GET_RECALL.

4. Part 2 — Three event-driven agent processes

Planner reads s3539_inbox and sends a plan to s3539_tasks. Writer reads s3539_tasks and sends a draft to s3539_drafts. Reviewer reads s3539_drafts and sends the approved JSON to s3539_final. Each stage preserves correlation_id entity_key question and the accumulated journey array.

4.1 Stage logic

```
ROLE_TOPICS = {
    "planner": (settings.inbox, settings.tasks),
    "writer": (settings.tasks, settings.drafts),
    "reviewer": (settings.drafts, settings.final),
}


def planner_answer(question: str) -> str:
    return (
        "1. Identify the requested decision and constraints. "
        "2. Collect the relevant grocery-recall facts. "
        "3. Draft a concise answer with consumer guidance."
    )


def writer_answer(question: str, plan: str) -> str:
    return (
        f"Question: {question}\n"
        "Draft: Review the recall code, affected product, and official consumer guidance. "
        "Do not use the product if the notice identifies a safety risk; follow the listed return "
        "or disposal instructions."
    )


def reviewer_answer(draft: str) -> str:
    return f"Approved: {draft}"


def process_stage(role: str, message: dict[str, Any]) -> dict[str, Any]:
    if role not in ROLE_TOPICS:
        raise ValueError(f"Unknown role: {role}")
    correlation_id = message.get("correlation_id")
    if not correlation_id:
        raise ValueError("correlation_id is required")
    question = str(message.get("question", "")).strip()
    if not question:
        raise ValueError("question is required")

    output = deepcopy(message)
    if role == "planner":
        output["plan"] = planner_answer(question)
        output["status"] = "planned"
    elif role == "writer":
        if not output.get("plan"):
            raise ValueError("plan is required")
        output["draft"] = writer_answer(question, output["plan"])
        output["status"] = "drafted"
    else:
        if not output.get("draft"):
            raise ValueError("draft is required")
        output["answer"] = reviewer_answer(output["draft"])
        output["status"] = "approved"
    output.setdefault("journey", []).append(role)
    return output
```

4.2 Separate process entry points

```
from code.hw8.agent_worker import run

if __name__ == "__main__":
    run("planner")
```

```
from code.hw8.agent_worker import run
```

```

if __name__ == "__main__":
    run("writer")
```

```
from code.hw8.agent_worker import run

if __name__ == "__main__":
    run("reviewer")
```

4.3 Live end-to-end message journey

```
PLANNER -> WRITER -> REVIEWER
Vedant Vartak - DATA 260 Homework 8
timestamp_utc=2026-10-27T05:13:28.012490+00:00

Vedant Vartak - DATA 260 Homework 8 three-agent pipeline
[USER SEND] topic=s3539_inbox partition=0 offset=4 message={"correlation_id": "f608cd81-02e6-400c-a9bf-
  dc77ee316f1e", "entity_key": "recall-3539", "journey": [], "question": "What should a customer do after
  finding a matching grocery recall?"}
[USER FINAL] topic=s3539_final correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e message={"answer": "Approved:
   Question: What should a customer do after finding a matching grocery recall?\nDraft: Review the recall code,
  affected product, and official consumer guidance. Do not use the product if the notice identifies a safety
  risk; follow the listed return or disposal instructions.", "correlation_id": "f608cd81-02e6-400c-a9bf-
  dc77ee316f1e", "draft": "Question: What should a customer do after finding a matching grocery recall?\nDraft:
  Review the recall code, affected product, and official consumer guidance. Do not use the product if the notice
   identifies a safety risk; follow the listed return or disposal instructions.", "entity_key": "recall-3539",
  "journey": ["planner", "writer", "reviewer"], "plan": "1. Identify the requested decision and constraints. 2.
  Collect the relevant grocery-recall facts. 3. Draft a concise answer with consumer guidance.", "question":
  "What should a customer do after finding a matching grocery recall?", "status": "approved"}
PASS journey=inbox -> tasks -> drafts -> final
```

Figure 4. Live question sent to s3539_inbox and final approved JSON received from s3539_final with journey [planner writer reviewer].

```
LIVE THREE-PROCESS STAGE LOGS
Vedant Vartak - DATA 260 Homework 8
reviewer-1       | [REVIEWER RECEIVE] topic=s3539_drafts partition=0 offset=44
  correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e
reviewer-1       | [REVIEWER SEND] topic=s3539_final partition=0 offset=44
  correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e status=approved
planner-1        | [PLANNER RECEIVE] topic=s3539_inbox partition=0 offset=4
  correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e
planner-1        | [PLANNER SEND] topic=s3539_tasks partition=0 offset=36
  correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e status=planned
writer-1-1       | [WRITER RECEIVE] topic=s3539_tasks partition=0 offset=36
  correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e
writer-1-1       | [WRITER SEND] topic=s3539_drafts partition=0 offset=44
  correlation_id=f608cd81-02e6-400c-a9bf-dc77ee316f1e status=drafted
```

Figure 5. Matching correlation ID in Planner Writer and Reviewer receive/send lines proves the separate-process journey.

The complete message includes the original question plan draft answer status correlation ID and entity key. This makes the full journey auditable without direct process-to-process calls.

5. Part 3A — Duplicate delivery and idempotency

Failure: the same CREATE message may be delivered more than once. Without idempotency this creates duplicate entities. Fix: the recall service caches the completed response by correlation_id. A replay returns the original entity_id and adds duplicate=true without changing the entity dictionary.

Observed item	Live value
Correlation ID	live-duplicate-c4b2d4ec-6d5c-4487-8c56-8175d09b173b
First entity ID	9f76942e-2efd-4896-aae1-f5b8fe31cc49
Replay entity ID	9f76942e-2efd-4896-aae1-f5b8fe31cc49
Replay marker	True
Result	PASS — same entity returned

```
        if data["category"] not in VALID_CATEGORIES:
            return "category is invalid"
        units = data.get("units_affected")
        if isinstance(units, bool) or not isinstance(units, int) or units <= 0:
            return "units_affected must be a positive integer"
        if data.get("terms_accepted") is not True:
            return "terms_accepted must be true"
        return None

    def process(self, message: Any) -> dict[str, Any]:
        if not isinstance(message, dict):
            return self._error("", "Request must be a JSON object")
        correlation_id = message.get("correlation_id")
        if not isinstance(correlation_id, str) or not correlation_id:
            return self._error("", "correlation_id is required")
        if correlation_id in self.responses_by_correlation:
            replay = deepcopy(self.responses_by_correlation[correlation_id])
            replay["duplicate"] = True
            return replay
        if not isinstance(message.get("reply_to"), str) or not message["reply_to"]:
            return self._error(correlation_id, "reply_to is required")

        operation = message.get("operation")
        data = message.get("data")
        if operation == "CREATE_RECALL":
            error = self.validate_create(data)
            if error:
                response = self._error(correlation_id, error)
            else:
                entity_id = str(uuid.uuid4())
                entity = {"entity_id": entity_id, **deepcopy(data)}
                self.entities[entity_id] = entity
                response = {
                    "correlation_id": correlation_id,
                    "success": True,
                    "entity_id": entity_id,
                    "message": "Recall created",
                }
        elif operation == "GET_RECALL":
```

6. Part 3B — Consumer killed mid-work

Failure: an auto-committed task can disappear if the Writer dies after receiving it but before publishing the draft. Fix: enable_auto_commit is false and the worker commits only after the next-topic send succeeds. The live experiment sends an eight-second Writer task stops both Writer containers while it is in flight restarts them and observes an approved final response.

```
"""Reusable Kafka process for the Planner, Writer, or Reviewer stage."""

from __future__ import annotations

import asyncio
import json
from code.hw8.config import settings
from code.hw8.pipeline import ROLE_TOPICS, process_stage
from typing import Any


async def run_agent_worker(role: str, *, max_retries: int = 3) -> None:
    from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

    input_topic, output_topic = ROLE_TOPICS[role]
    consumer = AIOKafkaConsumer(
        input_topic,
        bootstrap_servers=settings.kafka_bootstrap,
        group_id=f"{settings.prefix}-{role}-group",
        auto_offset_reset="earliest",
        enable_auto_commit=False,
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )
    producer = AIOKafkaProducer(
        bootstrap_servers=settings.kafka_bootstrap,
        value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    )
    processed: dict[str, dict[str, Any]] = {}
    await consumer.start()
    await producer.start()
    print(
        f"Vedant Vartak - {role.title()} ready: {input_topic} -> {output_topic}",
        flush=True,
    )
    try:
        async for record in consumer:
            message = record.value
            correlation_id = str(message.get("correlation_id", ""))
            print(
                f"[{role.upper()} RECEIVE] topic={record.topic} partition={record.partition} "
                f"offset={record.offset} correlation_id={correlation_id}",
                flush=True,
            )
            if correlation_id in processed:
                output = processed[correlation_id]
                print(f"[{role.upper()} IDEMPOTENT] replay ignored; reusing output", flush=True)
            else:
                output = None
                last_error = "unknown"
                for attempt in range(1, max_retries + 1):
                    try:
                        if message.get("simulate_delay_role") == role:
                            await asyncio.sleep(float(message.get("simulate_delay_seconds", 0)))
                        output = process_stage(role, message)
                        break
                    except ValueError as exc:
                        last_error = str(exc)
                        print(
                            f"[{role.upper()} RETRY] attempt={attempt}/{max_retries} "
                            f"correlation_id={correlation_id} error={last_error}",
                            flush=True,
                        )
                if output is None:
```

```
                    dead = {
                        "correlation_id": correlation_id,
                        "failed_role": role,
                        "error": last_error,
                        "original_message": message,
                    }
                    await producer.send_and_wait(settings.dead_letter, dead)
                    print(
                        f"[{role.upper()} DEAD LETTER] topic={settings.dead_letter} "
                        f"correlation_id={correlation_id}",
                        flush=True,
                    )
                    await consumer.commit()
                    continue
                processed[correlation_id] = output
            key = str(output.get("entity_key", correlation_id)).encode("utf-8")
            metadata = await producer.send_and_wait(output_topic, output, key=key)
            print(
                f"[{role.upper()} SEND] topic={metadata.topic} partition={metadata.partition} "
                f"offset={metadata.offset} correlation_id={correlation_id} "
                f"status={output['status']}",
                flush=True,
            )
            await consumer.commit()
            print(f"[{role.upper()} COMMIT] after_success=true", flush=True)
    finally:
        await consumer.stop()
        await producer.stop()


def run(role: str) -> None:
    asyncio.run(run_agent_worker(role))
```

```
LIVE FAILURE EXPERIMENTS
Vedant Vartak — DATA 260 Homework 8
PASS duplicate delivery: correlation_id=live-duplicate-c4b2d4ec-6d5c-4487-8c56-8175d09b173b
  first entity_id=9f76942e-2efd-4896-aae1-f5b8fe31cc49
  replay entity_id=9f76942e-2efd-4896-aae1-f5b8fe31cc49 duplicate=True
PASS consumer killed mid-work: correlation_id=live-restart-6e129273-8387-458c-81e0-a23f9965fa38
  both Writer containers stopped while an 8-second message was in flight
  restarted final status=approved journey=['planner', 'writer', 'reviewer']
PASS poison message: correlation_id=live-poison-716d8722-bf4a-4000-b060-ac26f16c02d0
  DLQ error=question is required
  later good status=approved
PASS all=True
```

Figure 6. Machine-readable summaries of the three live failure experiments. The full stop/start output offsets and messages are in live_failure_experiments.json.

7. Part 3C — Poison message with bounded retries

Failure: a blank question repeatedly fails Planner validation and can block the partition forever. Fix: each stage attempts processing at most three times then emits a dead-letter record with the original message failed_role correlation_id and error. It commits only after the DLQ publish. A later valid record then completes the full pipeline.

Evidence	Observed result
Poison correlation	live-poison-716d8722-bf4a-4000-b060-ac26f16c02d0
Attempts	3
DLQ topic	s3539_dead_letter
Error	question is required
Later message	approved after poison record

```
                        if message.get("simulate_delay_role") == role:
                            await asyncio.sleep(float(message.get("simulate_delay_seconds", 0)))
                        output = process_stage(role, message)
                        break
                    except ValueError as exc:
                        last_error = str(exc)
                        print(
                            f"[{role.upper()} RETRY] attempt={attempt}/{max_retries} "
                            f"correlation_id={correlation_id} error={last_error}",
                            flush=True,
                        )
                if output is None:
                    dead = {
                        "correlation_id": correlation_id,
                        "failed_role": role,
                        "error": last_error,
                        "original_message": message,
                    }
                    await producer.send_and_wait(settings.dead_letter, dead)
                    print(
                        f"[{role.upper()} DEAD LETTER] topic={settings.dead_letter} "
                        f"correlation_id={correlation_id}",
                        flush=True,
                    )
                    await consumer.commit()
                    continue
                processed[correlation_id] = output
            key = str(output.get("entity_key", correlation_id)).encode("utf-8")
            metadata = await producer.send_and_wait(output_topic, output, key=key)
            print(
                f"[{role.upper()} SEND] topic={metadata.topic} partition={metadata.partition} "
                f"offset={metadata.offset} correlation_id={correlation_id} "
                f"status={output['status']}",
                flush=True,
            )
            await consumer.commit()
            print(f"[{role.upper()} COMMIT] after_success=true", flush=True)
    finally:
        await consumer.stop()
        await producer.stop()


def run(role: str) -> None:
```

The service logs show PLANNER RETRY attempt=1/3 2/3 and 3/3 followed by PLANNER DEAD LETTER. They also show the next message being planned committed drafted reviewed and finalized.

8. Part 3D — Ordering across two partitions and two Writers

Using VERIFY_SEED=263539 the script sends 20 numbered messages through s3539_tasks with two partitions and two Writer processes. Alternating deterministic delays make unkeyed work complete out of order. The corrected run uses one entity_key as the Kafka key. Kafka hashes every related message to partition 0 and preserves their input order.

```
20-MESSAGE ORDERING EXPERIMENT
Vedant Vartak - DATA 260 Homework 8
VERIFY_SEED=263539
INPUT  = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
UNKEYED= [6, 8, 1, 2, 9, 3, 4, 15, 5, 17, 19, 7, 10, 20, 11, 12, 13, 14, 16, 18]
Result: related messages completed out of order across two partitions/two Writers.
KEYED  = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
KEYED PARTITIONS = [0]
PASS: entity_key kept all related messages on one partition and preserved order.
```

Figure 7. Required 20-message experiment: unkeyed completion is reordered; keyed completion is exactly 1 through 20 on one partition.

Case	Processed order	Result
Unkeyed	6, 8, 1, 2, 9, 3, 4, 15, 5, 17, 19, 7, 10, 20, 11, 12, 13, 14, 16, 18	Out of order
Keyed	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20	In order

8.1 Experiment implementation

```
async def consumer_death_case() -> dict[str, Any]:
    producer, final_consumer = await producer_and_consumer(settings.final, "restart-final")
    correlation_id = f"live-restart-{uuid.uuid4()}"
    message = {
        "correlation_id": correlation_id,
        "entity_key": "recall-restart-3539",
        "question": "Explain restart recovery.",
        "plan": "Produce one restart-safe draft.",
        "journey": ["planner"],
        "status": "planned",
        "simulate_delay_role": "writer",
        "simulate_delay_seconds": 8,
    }
    try:
        await producer.send_and_wait(settings.tasks, message, key=b"recall-restart-3539")
        await asyncio.sleep(2)
        stop_output = compose("stop", "writer-1", "writer-2")
        await asyncio.sleep(1)
        start_output = compose("start", "writer-1", "writer-2")
        final = (await matching(final_consumer, {correlation_id}, timeout=35))[0]
        return {
            "correlation_id": correlation_id,
            "stop_output": stop_output.strip(),
            "start_output": start_output.strip(),
            "final": final,
            "passed": final["value"]["status"] == "approved",
        }
    finally:
...
async def ordering_case() -> dict[str, Any]:
    producer, drafts = await producer_and_consumer(settings.drafts, "ordering")
    try:
        unkeyed_ids: list[str] = []
        keyed_ids: list[str] = []
        for sequence in range(1, 21):
            correlation_id = f"live-unkeyed-{uuid.uuid4()}"
            unkeyed_ids.append(correlation_id)
            await producer.send_and_wait(
                settings.tasks,
                {
                    "correlation_id": correlation_id,
                    "entity_key": f"unkeyed-{sequence}",
                    "sequence": sequence,
                    "question": "Ordering test",
                    "plan": "Preserve sequence evidence.",
                    "journey": ["planner"],
                    "simulate_delay_role": "writer",
                    "simulate_delay_seconds": 0.08 if sequence % 2 else 0.0,
                },
            )
        unkeyed_records = await matching(drafts, set(unkeyed_ids), timeout=35)
        unkeyed_order = [record["value"]["sequence"] for record in unkeyed_records]

        for sequence in range(1, 21):
            correlation_id = f"live-keyed-{uuid.uuid4()}"
            keyed_ids.append(correlation_id)
            await producer.send_and_wait(
                settings.tasks,
                {
                    "correlation_id": correlation_id,
                    "entity_key": "related-recall-3539",
                    "sequence": sequence,
                    "question": "Ordering test",
                    "plan": "Preserve sequence evidence.",
                    "journey": ["planner"],
                    "simulate_delay_role": "writer",
                    "simulate_delay_seconds": 0.01,
                },
                key=b"related-recall-3539",
            )
        keyed_records = await matching(drafts, set(keyed_ids), timeout=35)
```

```
        keyed_records.sort(key=lambda record: record["offset"])
        keyed_order = [record["value"]["sequence"] for record in keyed_records]
        keyed_partitions = sorted({record["partition"] for record in keyed_records})
        return {
            "verify_seed": settings.verify_seed,
            "input_order": list(range(1, 21)),
            "unkeyed_processed_order": unkeyed_order,
            "keyed_processed_order": keyed_order,
            "keyed_partitions": keyed_partitions,
            "passed": keyed_order == list(range(1, 21)) and len(keyed_partitions) == 1,
        }
    finally:
        await drafts.stop()
        await producer.stop()


async def main() -> None:
    RAW.mkdir(parents=True, exist_ok=True)
    payload = {
        "student": "Vedant Vartak",
        "timestamp_utc": datetime.now(UTC).isoformat(),
        "duplicate_delivery": await duplicate_case(),
        "consumer_killed_mid_work": await consumer_death_case(),
        "poison_message": await poison_case(),
        "two_partitions_two_consumers": await ordering_case(),
    }
    payload["passed"] = all(
        value["passed"]
        for key, value in payload.items()
        if isinstance(value, dict) and "passed" in value
    )
```

9. Failure table and HW2 comparison

Scenario	Before	After
Duplicate delivery	Replay creates another recall	Correlation cache returns original entity
Consumer death	Work may be acknowledged too early	Commit occurs after downstream publish
Poison message	Infinite retries block progress	Three tries then dead-letter; next record continues
Ordering	Related records span partitions and reorder	Entity key pins records to one partition

The HW2 graph executed nodes inside one process. That design was simple and deterministic but a process failure affected the whole graph and scaling a single stage was difficult. Kafka decouples the stages. Planner Writer and Reviewer can restart or scale independently and the durable log supports replay. The cost is explicit operational responsibility for idempotency manual commits poison-message policy consumer groups and partition keys.

This design is intentionally at-least-once. It avoids lost work by delaying commits and prevents duplicate side effects through idempotency rather than claiming exactly-once behavior.

10. Tests verification and raw artifacts

```
FINAL VERIFICATION
Vedant Vartak — DATA 260 Homework 8
commit_hash=66ef001731bf57b75530c0c1da2d55ca8153e0a1
SID4=3539 VERIFY_SEED=263539
PASS CREATE_RECALL followed by GET_RECALL round trip — entity_id=4bb77abe-a870-4ebd-823e-85b83b6968c7
PASS Planner Writer Reviewer correlation flow — inbox -> tasks -> drafts -> final
PASS Duplicate restart poison and ordering experiments — all passed
PASS Live Kafka failure experiments — live Kafka: duplicate restart poison ordering all passed
PASS Homework 8 unit suite — tests_run=18
PASS verification=True
```

Figure 8. Final verification at implementation commit 66ef001731bf. All five checks pass including 18 unit tests and direct inspection of the live failure JSON.

Test coverage includes exact topic names personal configuration successful CREATE/GET correlation preservation unknown entity unknown operation missing fields invalid units invalid email complete three-stage journey poison rejection idempotency manual-commit behavior dead-letter progress deterministic ordering and all four combined experiments.

The raw artifacts are not reconstructed from report prose. They are saved independently:

Artifact	Purpose
raw/homework_demo_live.txt	Live CREATE/GET inputs and outputs
raw/agent_pipeline_live.txt	Live inbox-to-final JSON
raw/kafka_service_logs.txt	Topic creation worker retries commits and sends
raw/live_failure_experiments.json	All live failure inputs outputs and PASS fields
raw/ordering_20_messages.csv	Twenty-record unkeyed/keyed comparison
verification.json	Configuration commit hash tests and pass/fail summary

11. Supplemental JMeter plan

The supplied JMeter instructions are supplemental to the Kafka requirements. I ran reports/hw08/jmeter/HW8_direct_100_users.jmx against POST /api/hw8/entities-direct with 100 threads a 10-second ramp JSON headers unique per-thread correlation IDs and an HTTP 201 assertion. This isolates direct FastAPI capacity from Kafka round-trip latency as requested by that handout.



Figure 9. Real JMeter non-GUI run: 100 samples 100 passed 0% errors average 4.04 ms and 10.13 requests per second during the configured ramp.

Samples	Passed	Error %	Average	Min	Max	p95	Throughput
100	100	0.00	4.04 ms	1 ms	15 ms	8 ms	10.13 req/s

The editable plan is saved at reports/hw08/jmeter/HW8_direct_100_users.jmx. Raw per-request samples are saved in reports/hw08/raw/jmeter_100_users.jtl and the summarized values are in jmeter_summary.json.

12. Reproduction commands

```
make hw8-up
.venv/bin/python -m code.hw8.homework_demo
.venv/bin/python -m code.hw8.send_question
.venv/bin/python scripts/run_hw08_live_failures.py
make experiment-hw8
make test-hw8
make verify-hw08
make hw8-down
```

The live failure script is destructive only to the two Writer containers inside the dedicated data260-hw8 Compose project. It stops and restarts them as part of the required recovery demonstration. No database from another homework is modified.

13. AI-use disclosure

1. I used an AI assistant to help debug code and organize the report.

2. I independently checked the required topic names and correlation ID flow against the assignment.

3. I found mistakes by running the automated tests and comparing the saved JSON logs with the requirements.

4. I corrected the message validation and added bounded retry idempotency and ordering checks.

14. Submission contents

The hw8 tag contains the implementation Docker configuration scripts tests report raw evidence CSV JMeter plan METRICS.md RUN_LOG.txt AI_USE.md and verification.json. The report filename is Vartak_HW8.pdf.